

Pomagalo za gRPC (1. zadatak)

// UNARNA — 1 ulaz, 1 izlaz

```
public override Task<Res> M(Req request, ServerCallContext context)
```

// SERVER STREAMING — 1 ulaz, tok izlaza (Task, BEZ <Res>!)

```
public override async Task M(Req request,  
    IServerStreamWriter<Res> responseStream, ServerCallContext  
    context)
```

// CLIENT STREAMING — tok ulaza, 1 izlaz

```
public override async Task<Res> M(  
    IAsyncStreamReader<Req> requestStream, ServerCallContext  
    context)
```

// BIDIREKCIONALNA — tok ulaza, tok izlaza (Task, BEZ <Res>!)

```
public override async Task M(IAsyncStreamReader<Req>  
    requestStream,  
    IServerStreamWriter<Res> responseStream, ServerCallContext  
    context)
```

// Kada se cita CLIENT STREAMING

```
await foreach (var b in requestStream.ReadAllAsync()) // čitaj dok  
    klijent šalje
```

// Kada se salje SERVER STREAMING

```
foreach (var m in messages.Values) await response.WriteAsync(m);
```

Primer RMI (2. zadatak)

Server.java

```
public class Server {  
    public static void main(String[] args) {  
        try {  
            LocateRegistry.createRegistry(1099);  
        } catch (RemoteException e) {  
            System.out.println("Registry vec postoji.");  
        }  
  
        try {  
            Broker broker = new BrokerImpl();  
            Naming.rebind("rmi://localhost/BrokerService", broker);  
            System.out.println("Server startovan...");  
        } catch (Exception e) {  
            e.printStackTrace();  
        }  
    }  
}
```

Message.java

```
public class Message implements Serializable {  
    private String naslov;  
    private String poruka;  
  
    public Message(String naslov, String poruka) {  
        this.naslov = naslov;  
        this.poruka = poruka;  
    }  
  
    public String getNaslov() { return naslov; }  
    public String getPoruka() { return poruka; }  
  
    public String toString() {  
        return naslov + " " + poruka;  
    }  
}
```

Client.java

```
public class Client {
    public static void main(String[] args) {
        try {
            Broker broker = (Broker)
Naming.lookup("rmi://localhost/BrokerService");

            ClientCallbackImpl clb = new ClientCallbackImpl();
            broker.subscribe("sport", clb);

            Scanner sc = new Scanner(System.in);
            while (true) {
                System.out.print("Naslov (ili 'exit'): ");
                String naslov = sc.nextLine();
                if (naslov.equalsIgnoreCase("exit")) break;
                System.out.print("Sadrzaj: ");
                String sadrzaj = sc.nextLine();
                broker.publish("sport", new Message(naslov, sadrzaj));
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    public static class ClientCallbackImpl extends UnicastRemoteObject
implements ClientCallback {
        public ClientCallbackImpl() throws RemoteException { super(); }

        public void notify(String topic, Message msg) throws
RemoteException {
            System.out.println("Stiglo obavestenje [" + topic + "]: " + msg);
        }
    }
}
```

Broker.java

```
public interface Broker extends Remote {  
    void subscribe(String topic, ClientCallback clb) throws RemoteException;  
    void publish(String topic, Message msg) throws RemoteException;  
}
```

ClientCallback.java

```
public interface ClientCallback extends Remote {  
    void notify(String topic, Message msg) throws RemoteException;  
}
```

BrokerImpl.java

```
public class BrokerImpl extends UnicastRemoteObject implements Broker {  
  
    private final Map<String, Set<ClientCallback>> topics;  
  
    public BrokerImpl() throws RemoteException {  
        topics = new ConcurrentHashMap<>();  
    }  
  
    private void createTopic(String topic) {  
        topics.putIfAbsent(topic, new HashSet<>());  
    }  
  
    public void subscribe(String topic, ClientCallback clb) throws  
RemoteException {  
        createTopic(topic);  
        topics.get(topic).add(clb);  
        System.out.println("Klijent se pretplatio na temu " + topic);  
    }  
  
    public void publish(String topic, Message msg) throws RemoteException  
{  
        createTopic(topic);  
        for (ClientCallback c : topics.get(topic)) {  
            c.notify(topic, msg);  
        }  
    }  
}
```

Primer WCF (3. zadatak)

Web.Config

basicHttpBinding ide kada je komunikacija server-client

```
<configuration>
  <system.serviceModel>
    <services>
      <service name="Jun24.ZakupSkladista">
        <endpoint contract="Jun24.IZakupSkladista"
binding="basicHttpBinding" />
      </service>
    </services>
  </system.serviceModel>
</configuration>
```

wsDualHttpBinding ide kada je full-duplex, sa callback

```
<configuration>
  <system.serviceModel>
    <services>
      <service name="Okt225.Kalkulator">
        <endpoint contract="Okt225.IKalkulator" binding="wsDualHttpBinding"
/>
      </service>
    </services>
    <bindings>
      <wsDualHttpBinding>
        <binding name="wsDualHttpBindingConfiguration"
transactionFlow="true" />
      </wsDualHttpBinding>
    </bindings>
    <protocolMapping>
      <add scheme="http" binding="wsDualHttpBinding"
bindingConfiguration="wsDualHttpBindingConfiguration"/>
    </protocolMapping>
  </system.serviceModel>
</configuration>
```

Kalkulator.svc

```
[ServiceBehavior(InstanceContextMode =
InstanceContextMode.PerSession)]
public class Kalkulator : IKalkulator {
    private string koraci = "0";
    private double rezultat = 0;

    private IKalkulatorCallback callback;

    public Kalkulator() {
        callback =
OperationContext.Current.GetCurrentCallbackChannel<IKalkulatorCallback
>();
    }

    public double ObrisiRacunicu() {
        koraci = "0";
        rezultat = 0;

        callback.PrikaziIzraz(koraci);

        return rezultat;
    }
    ....
}
```

Ovo je svc kada imamo callback, bez callback je sve isto samo nema callback

IKalkulatorCallback.cs

```
public interface IKalkulatorCallback {
    [OperationContract(IsOneWay = true)]
    void PrikaziIzraz(string izraz);
}
```

Ovo kad ima callback, inace ne pisemo ovu klasu

IKalkulator.cs

```
[ServiceContract(CallbackContract = typeof(IKalkulatorCallback),
SessionMode = SessionMode.Required)]
public interface IKalkulator {
    [OperationContract]
    double ObrisiRacunicu();

    [OperationContract]
    [FaultContract(typeof(KalkulatorGreska))]
    Response PodeliRezultatBrojem(int broj);
}
```

Ako nemamo callback pisemo samo ServiceContract, a ovaj faultcontract ako se trazi pisemo ovako

KalkulatorGreska.cs

```
[DataContract]
public class KalkulatorGreska {
    [DataMember]
    public string Poruka { get; set; }
}
```

Pravimo klasu Greska kada imamo Fault da se trazi, inace klase normalno pisemo ovako sa DataContract i DataMember

```
if(broj == 0) {
    throw new FaultException<KalkulatorGreska>(
        new KalkulatorGreska { Poruka = "Deljenje nulom nije dozvoljeno " }
    );
}
```

Ovo je primer kada koristimo u kodu gresku da bacimo exception ako se trazi FaultException

Client.cs

```
public class KalkulatorCallback : IKalkulatorCallback {
    public void PrikaziIzraz(string izraz) {
        Console.WriteLine($"[callback] Izraz: {izraz}");
    }
}

public class Client {
    public static void Main(string[] args) {
        InstanceContext context = new InstanceContext(new
KalkulatorCallback());
        KalkulatorClient proxy = new KalkulatorClient(context);

        Console.WriteLine($"Rezultat: {proxy.DodajBroj(2)}");
        Console.WriteLine($"Rezultat: {proxy.DodajBroj(3)}");
        Console.WriteLine($"Rezultat: {proxy.OduzmiBroj(5)}");
        Console.WriteLine($"Rezultat: {proxy.PomnoziBrojem(7)}");
        Console.WriteLine($"Rezultat: {proxy.PodeliBroj(2)}");

        try {
            proxy.PodeliBroj(0);
        }
        catch (FaultException<KalkulatorGreska> ex) {
            Console.WriteLine($"Greska: {ex.Detail.Poruka}");
        }

        Console.WriteLine($"Rezultat: {proxy.ObrisiRacunicu()}");

        Console.ReadLine();
        proxy.Close();
    }
}
```

Ovaj client ima sva 3 primera, znaci i callback i fault i generalno kako se pise, znaci ako nema callback nemamo ni context ni KalkulatorCallback klasu, a ovaj try catch je samo kad imamo fault, tako da ovo je najkompletniji client, a u zavisnosti sta se trazi pravimo to i izbacujemo stvari

Primer MPI (4. zadatak)

jun 2024, zad. 3 (a + b)

a) upis 105 brojeva **pojedinačnim pokazivačima**, redosled u fajlu od poslednjeg do prvog procesa. **b)** ponovo otvoriti fajl i pročitati iste podatke **eksplicitnim pomerajem**.

```
#define N 105
```

```
int main(int argc, char** argv)
```

```
{
```

```
    int rank, size;
```

```
    MPI_File file1;
```

```
    MPI_Offset offset;
```

```
    char file1Path[] = "file1.dat";
```

```
    MPI_Init(&argc, &argv);
```

```
    MPI_Comm_size(MPI_COMM_WORLD, &size);
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
```

```
    int* inputBuf = (int*)malloc(N * sizeof(int));
```

```
    for (int i = 0; i < N; i++)
```

```
        inputBuf[i] = rank;
```

```
    /* unazad: poslednji proces ide na pocetak fajla */
```

```
    offset = (MPI_Offset)(size - 1 - rank) * N * sizeof(int);
```

```
    /* a) POJEDINACNI POKAZIVAC -> seek + write */
```

```
    MPI_File_open(MPI_COMM_WORLD, file1Path,
```

```
                  MPI_MODE_CREATE | MPI_MODE_WRONLY,
```

```
    MPI_INFO_NULL, &file1);
```

```
    MPI_File_seek(file1, offset, MPI_SEEK_SET);
```

```
    MPI_File_write(file1, inputBuf, N, MPI_INT, MPI_STATUS_IGNORE);
```

```
    MPI_File_close(&file1);
```

```
    free(inputBuf);
```

```
    MPI_Barrier(MPI_COMM_WORLD);
```

```

/* b) EKSPPLICITNI POMERAJ -> read_at, BEZ seek-a */
int* readBuf = (int*)malloc(N * sizeof(int));
MPI_File_open(MPI_COMM_WORLD, file1Path,
               MPI_MODE_RDONLY, MPI_INFO_NULL, &file1);
MPI_File_read_at(file1, offset, readBuf, N, MPI_INT,
MPI_STATUS_IGNORE);
MPI_File_close(&file1);

printf("Proces %d procitao: %d %d ...\n", rank, readBuf[0], readBuf[1]);

free(readBuf);
MPI_Finalize();
return 0;
}

```

jun 2026, zad. 18 (prvi deo)

Datoteka input.dat ima 1kB. Svi procesi čitaju istu količinu podataka funkcijama sa eksplcitnim pomerajem, tako da prvi proces čita poslednji skup, drugi preposlednji, itd.

```
#define FILESIZE 1024
```

```

int main(int argc, char** argv)
{
    int rank, size;
    MPI_File file1;
    MPI_Offset offset;
    char file1Path[] = "input.dat";

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int totalInts = FILESIZE / sizeof(int);
    int localInts = totalInts / size;          /* elemenata po procesu */
    int* buf = (int*)malloc(localInts * sizeof(int));

```

```

/* unazad: prvi proces cita poslednji skup */
offset = (MPI_Offset)(size - 1 - rank) * localInts * sizeof(int);

/* EKSPLICITNI POMERAJ -> read_at, BEZ seek-a */
MPI_File_open(MPI_COMM_WORLD, file1Path,
               MPI_MODE_RDONLY, MPI_INFO_NULL, &file1);
MPI_File_read_at(file1, offset, buf, localInts, MPI_INT,
MPI_STATUS_IGNORE);
MPI_File_close(&file1);

MPI_Barrier(MPI_COMM_WORLD);

printf("Proces %d procitao %d brojeva, prvi = %d\n", rank, localInts,
buf[0]);

free(buf);
MPI_Finalize();
return 0;
}

```

okt2 2025, zad. 17 (prvi deo)

Datoteka `input.dat` ima 10MB. Svi procesi čitaju istu količinu podataka pojedinačnim pokazivačem, tako da poslednji proces čita prvi skup, pretposlednji drugi, itd. Koristi se osmobitni tip podataka.

```
#define FILESIZE 10*1024*1024
```

```
int main(int argc, char** argv)
```

```
{
```

```
    int rank, size;
```

```
    MPI_File file1;
```

```
    MPI_Offset offset;
```

```
    char file1Path[] = "input.dat";
```

```
    MPI_Init(&argc, &argv);
```

```
    MPI_Comm_size(MPI_COMM_WORLD, &size);
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
```

```
    int bufsz = FILESIZE / size;          /* bajtova po procesu */
```

```
    char* buf = (char*)malloc(bufsz);     /* osmobitni tip -> char */
```

```
    /* unazad: poslednji proces cita prvi skup */
```

```
    offset = (MPI_Offset)(size - 1 - rank) * bufsz * sizeof(char);
```

```
    /* POJEDINACNI POKAZIVAC -> seek + read */
```

```
    MPI_File_open(MPI_COMM_WORLD, file1Path,
```

```
                  MPI_MODE_RDONLY, MPI_INFO_NULL, &file1);
```

```
    MPI_File_seek(file1, offset, MPI_SEEK_SET);
```

```
    MPI_File_read(file1, buf, bufsz, MPI_CHAR, MPI_STATUS_IGNORE);
```

```
    MPI_File_close(&file1);
```

```
    MPI_Barrier(MPI_COMM_WORLD);
```

```
    printf("Proces %d procitao %d bajtova\n", rank, bufsz);
```

```
    free(buf);
```

```
    MPI_Finalize();
```

```
    return 0;
```

```
}
```